EI1024/MT1024 "Programación Concurrente y Paralela" 2025–26	Entregable para Laboratorio
Nombre y apellidos (1): Miguel Ruiz Tena	
Nombre y apellidos (2):	
Tiempo empleado para tareas en casa en formato *h:mm* (obligatorio): 0:50	la03_g

Tema 04. El Problema de la Visibilidad en Java

Tema 05. El Problema de la Atomicidad en Java

1 Estudia el siguiente código y responde a las siguientes preguntas.

```
// ============================================================================
class CuentaIncrementos {
    // ========================================================================
    int numIncrementos = 0;

    // ------------------------------------------------------------------------
    void incrementaNumIncrementos() {
        numIncrementos++;
    }

    // ------------------------------------------------------------------------
    int dameNumIncrementos() {
        return( numIncrementos );
    }
}


// ============================================================================
class MiHebra extends Thread {
    // ========================================================================
    int                 numIters;
    CuentaIncrementos   c;

    // ------------------------------------------------------------------------
    public MiHebra( int numIters, CuentaIncrementos c ) {
        this.numIters = numIters;
        this.c        = c;
    }

    // ------------------------------------------------------------------------
    public void run() {
        for( int i = 0; i < numIters; i++ ) {
            c.incrementaNumIncrementos();
        }
    }
}

// ============================================================================
class EjemploCuentaIncrementos {
    // ========================================================================

    // ------------------------------------------------------------------------
```

```
    public static void main( String args[] ) {
        long     t1, t2;
        double   tt;
        int      numHebras, numIters;

        // Comprobacion y extraccion de los argumentos de entrada.
        if( args.length != 2 ) {
            System.err.println( "Uso: java programa <numHebras> <numIters>" );
            System.exit( -1 );
        }
        try {
            numHebras = Integer.parseInt( args[ 0 ] );
            numIters  = Integer.parseInt( args[ 1 ] );
            if( ( numHebras <= 0 ) || ( numIters <= 0 ) ) {
                System.err.print( "Uso: [ java programa <numHebras> <n> ] " );
                System.err.println( "donde ( numHebras > 0 )  y ( numIters > 0 )" );
                System.exit( -1 );
            }
        } catch( NumberFormatException ex ) {
            numHebras = -1;
            numIters  = -1;
            System.out.println( "ERROR: Argumentos numericos incorrectos." );
            System.exit( -1 );
        }

        System.out.println( "numHebras: " + numHebras );
        System.out.println( "numIters : " + numIters );

        System.out.println( "Creando y arrancando " + numHebras + " hebras." );
        t1 = System.nanoTime();
        MiHebra v[] = new MiHebra[ numHebras ];
        CuentaIncrementos c = new CuentaIncrementos();
        for( int i = 0; i < numHebras; i++ ) {
            v[ i ] = new MiHebra( numIters, c );
            v[ i ].start();
        }
        for( int i = 0; i < numHebras; i++ ) {
            try {
                v[ i ].join();
            } catch( InterruptedException ex ) {
                ex.printStackTrace();
            }
        }
        t2 = System.nanoTime();
        tt = ( ( double ) ( t2 - t1 ) ) / 1.0e9;
        System.out.println( "Total de incrementos: " + c.dameNumIncrementos() );
        System.out.println( "Tiempo transcurrido en segs.: " + tt );
    }
}
```

1.1) ¿Qué realiza el código? ¿Qué debería mostrar en pantalla si se ejecutase con los parámetros hebras 4 y numIters 1 000 000?

El código incrementa una variable numIter veces por cada hebra, es decir, en este caso incrementa la variable hasta 4 millones

1.2) Compila y ejecuta el código con dichos valores en tu ordenador local. ¿Qué muestra realmente en pantalla si se ejecuta con los parámetros hebras 4 y numIters 1 000 000?

Muestra solo 1389703 incrementos

1.3) ¿Es un código *thread-safe*? Justifica tu respuesta.

No ya que el objeto c no es atómico

1.4) Crea una **copia del código original** (EjemploCuentaIncrementosVolatile.java) e inserta el modificador `volatile` en la variable `numIncrementos` de la clase `CuentaIncrementos`.
A continuación, compila y prueba el nuevo código.
¿Resuelve el problema el modificador `volatile`? ¿Por qué?

No lo resuelve ya que el principal problema es de atomicidad, no de visibilidad

1.5) ¿Se podría resolver con el modificador `synchronized`?
Para comprobarlo, crea una **copia del código original** (EjemploCuentaIncrementosSynchronized.java) y aplica el modificador `synchronized` sobre **cada una** de las rutinas de la clase `CuentaIncrementos`.
Después, compila y prueba el código, antes de contestar a la pregunta anterior.
Escribe a continuación los cambios realizados en la clase `CuentaIncrementos`.

```
// ================================================================
class CuentaIncrementos {
//-----------------------------------------------------------------
  int numIncrementos = 0;

  // ---------------------------------------------------------------
  // Se añade el modificador "synchronized"
  synchronized void incrementaNumIncrementos() {
    numIncrementos++;
  }

  //----------------------------------------------------------------
  // Se añade el modificador "synchronized"
  synchronized int dameNumIncrementos() {
    return( numIncrementos );
  }
}
```

Si, resuelve los problemas

1.6) ¿También se podría arreglar empleando clases y operadores atómicos?
Para comprobarlo, crea otra **copia del código original** (EjemploAtomic.java), **ELIMINA** la clase `CuentaIncrementos` y utiliza en su lugar una **clase atómica y sus métodos**.
Después, compila y prueba el código, antes de contestar la pregunta.
Escribe a continuación los cambios realizados en el código.

Código adjunto en EjemploAtomic.java

Si, tambien funciona

1.7) Completa la siguiente tabla con datos de todas las versiones anteriores en tu ordenador, utilizando hebras 4 y un numIters de 1 000 000. Comenta los resultados.

Código	Total incrementos
Código original	1389703
Código con `volatile`	1799670
Código con `synchronized`	4000000
Código con clases atómicas	4000000

Podemos observar que tanto las clases atómicas como el uso de synchronzied soluciona el problema.

Por otro lado, vemos que tanto el original como el que usa volatile no llegan ni a la mitad de los incrementos

2 Se desea imprimir en pantalla los números primos que aparecen en un vector.

El código completo es el siguiente:

```
// ============================================================================
public class EjemploMuestraPrimosEnVector {
// ============================================================================

  // --------------------------------------------------------------------------
  public static void main( String args[] ) {
    int     numHebras, vectOpt;
    boolean option = true;
    long    t1, t2;
    double  ts, tc, tb, td;

    // Comprobacion y extraccion de los argumentos de entrada.
    if( args.length != 2 ) {
      System.err.println( "Uso: java programa <numHebras> <vectOpt>" );
      System.exit( -1 );
    }
    try {
      numHebras = Integer.parseInt( args[ 0 ] );
      vectOpt   = Integer.parseInt( args[ 1 ] );
      if( ( numHebras <= 0 ) || ( ( vectOpt != 0 ) && ( vectOpt != 1 ) ) ){
          System.err.print( "Uso: [ java programa <numHebras> <vecOpt> ] " );
          System.err.println( "donde ( numHebras > 0 )  y ( vectOpt es 0 o 1 )" );
          System.exit( -1 );
      } else {
        option = (vectOpt == 0);
      }
    } catch( NumberFormatException ex ) {
      numHebras = -1;
      System.out.println( "ERROR: Argumentos numericos incorrectos." );
      System.exit( -1 );
    }

    //
    // Eleccion del vector de trabajo
    //
    VectorNumeros vn = new VectorNumeros (option);
    long vectorTrabajo[] = vn.vector;

    //
    // Implementacion secuencial.
    //
    System.out.println( "" );
    System.out.println( "Implementacion secuencial." );
    t1 = System.nanoTime();
    for( int i = 0; i < vectorTrabajo.length; i++ ) {
      if( esPrimo( vectorTrabajo[ i ] ) ) {
        System.out.println( "  Encontrado primo: " + vectorTrabajo[ i ] );
      }
    }
    t2 = System.nanoTime();
    ts = ( ( double ) ( t2 - t1 ) ) / 1.0e9;
    System.out.println( "Tiempo secuencial (seg.):                    " + ts );
/*
    //
    // Implementacion paralela ciclica.
    //
    System.out.println( "" );
    System.out.println( "Implementacion paralela ciclica." );
```

```
      t1 = System.nanoTime();
      // Gestion de hebras para la implementacion paralela ciclica
      // (A) ....
      t2 = System.nanoTime();
      tc = ( ( double ) ( t2 - t1 ) ) / 1.0e9;
      System.out.println( "Tiempo paralela ciclica (seg.):          " + tc );
      System.out.println( "Incremento paralela ciclica:             " + ... ); // (B)
      //
      // Implementacion paralela por bloques.
      //
      // (C) ....
      //
      // Implementacion paralela dinamica.
      //
      // (D) ....
*/
  }

  // ---------------------------------------------------------------------
  static boolean esPrimo( long num ) {
    boolean cond;
    if( num < 2 ) {
      cond = false;
    } else {
      cond = true;
      long i = 2;
      while( ( i < num )&&( cond ) ) {
        cond = ( num % i != 0 );
        i++;
      }
    }
    return( cond );
  }
}

// Definicion de las Clases Hebras
//
// (E) ....

// =====================================================================
class VectorNumeros {
// =====================================================================
  long     vector[];
  // -------------------------------------------------------------------
  public VectorNumeros (boolean caso) {
    if (caso) {
      vector = new long [] {
      200000081L, 200000083L, 200000089L, 200000093L,
      200000107L, 200000117L, 200000123L, 200000131L,
      200000161L, 200000183L, 200000201L, 200000209L,
      200000221L, 200000237L, 200000239L, 200000243L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
```

```
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L
        };
    } else {
        vector = new long [] {
        200000081L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000083L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000089L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000093L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000107L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000117L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000123L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000131L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000161L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000183L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000201L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000209L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000221L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000237L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000239L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        200000243L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L
        };
    }
  }
}
```

2.1) Compila y ejecuta el programa anterior, utilizando un 0 como segundo parámetro. En este caso se trabaja con el siguiente vector de números:

```
long vectorTrabajo[] = {
        200000081L, 200000083L, 200000089L, 200000093L,
        200000107L, 200000117L, 200000123L, 200000131L,
        200000161L, 200000183L, 200000201L, 200000209L,
        200000221L, 200000237L, 200000239L, 200000243L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
        4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L
    };
```

¿Cuáles son los números primos contenidos en el vector?

200000081L , 200000083L , 200000089L , 200000093L ,
200000107L , 200000117L , 200000123L , 200000131L ,
200000161L , 200000183L , 200000201L , 200000209L ,
200000221L , 200000237L , 200000239L , 200000243L

2.2) Realiza una implementación paralela con distribución cíclica, en la que cada hebra procese un conjunto de elementos del vector. Para cada elemento del vector procesado, SOLO se mostrará su valor si el número es primo.

Descomenta el código situado debajo de"Implementacion secuencial". Incluye la gestión de hebras que paraleliza el bucle comprendido entre la lectura de t1 y t2 en la versión secuencial, y la expresión que permite calcular el incremento de velocidad.

Comprueba que los números primos mostrados en la versión paralela coinciden con los de la versión secuencial.

Escribe, a continuación, la parte de tu código que realiza tal tarea: la definición de la clase `MiHebraPrimoDistCiclica` (E) y el código a incluir en el programa principal que permite gestionar los objetos de esta clase (A-B).

Codigo adjunto en EjemploMuestraPrimosEnVector.java

ATENCIÓN: Los ejercicios anteriores deben realizarse en casa. Los siguientes, en el aula.

2.3) Realiza una implementación paralela con distribución por bloques, en la que cada hebra procese un conjunto de elementos del vector. Para cada elemento del vector procesado, SOLO se mostrará su valor si el número es primo.

Crea en (E) una clase nueva hebra para este caso. Replica en (C) el código del programa principal de la "Implementación paralela cíclica", para que se ejecute tras las otras versiones.

Comprueba que los números primos mostrados en la versión paralela coinciden con los de la versión secuencial.

Escribe, a continuación, la parte de tu código que realiza tal tarea: la definición de la clase `MiHebraPrimoDistPorBloques` (E) y el código a incluir en el programa principal que permite gestionar los objetos de esta clase (A-B).

Codigo adjunto en EjemploMuestraPrimosEnVector.java

2.4) Realiza una implementación paralela con distribución dinámica, que utilice un número entero atómico (`AtomicInteger`), para apuntar a una posición del vector. Las hebras recibirán un objeto de este tipo, que siempre contendrá la primera posición del vector sin procesar. Para ello, las hebras deben **realizar, de modo atómico, la lectura del valor actual y su incremento**. Las hebras finalizarán cuando el índice sobrepase la dimensión del vector.

Crea en (E) una clase nueva hebra para este caso. Replica en (D) el código del programa principal de la "Implementación paralela cíclica", para que se ejecute tras las otras versiones.

Comprueba que los números primos mostrados en la versión paralela coinciden con los de la versión secuencial.

Escribe, a continuación, la parte de tu código que realiza tal tarea: la definición de la clase `MiHebraPrimoDistDinamica` (E) y el código a incluir en el programa principal que permite gestionar los objetos de esta clase (A-B).

Codigo adjunto en EjemploMuestraPrimosEnVector.java

2.5) Completa la siguiente tabla, obteniendo los resultados para 4 hebras en el ordenador del aula y los resultados para 16 hebras en karen. Redondea los tiempos dejando sólo tres decimales y redondea los incrementos dejando dos decimales.

	4 hebras (aula)		16 hebras (karen)	
	Tiempo	Incremento	Tiempo	Incremento
Secuencial	23.455	—	16.023	—
Paralela con distribución cíclica	7.879	2.98	1.016	15.77
Paralela con distribución por bloques	28.922	0.81	16.007	1.00
Paralela con distribución dinámica	7.678	3.05	1.00	15.900

2.6) Justifica los resultados de la tabla anterior.

Observamos que tanto la distribución cíclica como la dinámica obtienen un incremento de alrededor de n-1 en local (aunque debería ser de n pero al parecer Windows hace de las suyas) y de n en karen, por otro lado podemos ver quedebido al orden del vector, la distribución por bloques incluso empeora el rendimiento de la sencuencial ya que realmente solo trabaja una hebra de las 4/16 creadas

2.7) Evalúa y compara las tres versiones (secuencial, paralela cíclica y paralela por bloques), pero en este caso utilizando 1 como segundo parámetro, es decir, manejando el vector:

```
long vectorTrabajo[] = {
      200000081L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000083L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000089L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000093L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000107L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000117L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000123L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000131L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000161L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000183L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000201L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000209L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000221L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000237L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000239L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L,
      200000243L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L, 4L
    };
```

Completa la siguiente tabla, obteniendo los resultados para 4 hebras en el ordenador del aula y los resultados para 16 hebras en karen. Redondea los tiempos dejando sólo tres decimales y redondea los incrementos dejando dos decimales.

	4 hebras (aula)		16 hebras (karen)	
	Tiempo	Incremento	Tiempo	Incremento
Secuencial	24.823	—	16.023	—
Paralela con distribución cíclica	25.328	0.98	16.009	1.00
Paralela con distribución por bloques	7.363	3.37	1.008	15.90
Paralela con distribución dinámica	7.984	3.11	1.006	15.93

2.8) Justifica los resultados de la tabla anterior.

Los resultados son prácticamente identicos a los anteriores cambiando el mal rendimiento anterior de la distribución por bloques a la distribución cíclica, ya que el nuevo orden beneficia a la primera pero perjudica a la segunda

2.9) ¿Cuál es la mejor distribución con ambos vectores? Justifica tu respuesta.

Ante los resultados de las pruebas anteriores, podemos concluir que la distribución dinámica es la mejor ya que no depende del orden de los datos, sino que se adapta a la carga de trabajo

3 Empleando el ordenador del aula, completa la siguiente tabla con datos de todas las versiones desarrolladas en el ejercicio 1, utilizando hebras 4 y un numIters de 10 000 000. Redondea los tiempos dejando sólo tres decimales y comenta los resultados.

Código	Total incrementos	Tiempo transcurrido (seg.)
Código original	19913976	0.020
Código con `volatile`	12612329	0.875
Código con `synchronized`	40000000	0.980
Código con clases atómicas	40000000	0.640

Similar al ejercicio 1, el código original y el que usa volatile, no funcionan correctamente, además, el volatile funciona extremadamente lento para hacer solo una cuarta parte de los incrementos que debe hacer

Por otro lado, la clase con synchronized y con clases atómicas, si realizan todos los incrementos que deben, siendo alrededor de un 33% más rapida la que usa clases atómicas